Electronic System Level Design Methodology

Project 1, 2026-03-26

Abstract

Explore Virtualizer, SystemC, TLM 2.0 and Multicore examples/demos. Then, evolve from these examples to port two (2) ARMv8 cores onto the Virtualizer and exchange data via DMA2 on shared memory.

Please read carefully. All outputs required are described in the text. Five (5) points will be taken for each missing required output and behavior.

Description

This is an individual project, with all classmates sharing their findings in the class Line group. An individual project with group efforts, so to speak.

PART I

1. There are ARMv8 multi-core examples in the VDK Creation Perspective in Virtualizer (vs). You can modify an ARMv8 project to complete the project.
2. Materials to study:
   A. TLM Creator User Guide – for porting your DMA module into vs.
   B. VDK Creator User Guide – for creating ARMv8 architectures
   C. TLMC_Training_x-2025.06 – for learning TLM Creator
   D. VDKCreation_Quickstart_Training_X-2025.06 – for learning VDK Creator
3. Ask ChatGPT, Gemini, or Codex for help loading the DMA2 module into the Virtualizer and integrating it with the ARMv8 architecture.

PART II

1. First, place two (2) ARMv8 cores, four (4) 1MB RAMs, a DMA2, two (2) UARTs, and three (3) TLM 2.0 buses into the system, with addresses and connections as described in Figure 1.
2. The DMA2 needs to be modified from the DMA of Assignment 1 to have two (2) interrupt output ports, `INT1` and `INT2`. There are two (2) sets of control registers; one set is addressed between `0x0` and `0xF`, and the second set is addressed between `0x10` and `0x1F`. Each set of control registers is defined in the same way as in Assignment 1. The first set is used by ARMv8_0 and the second by ARMv8_1. There is only one (1) slave port and

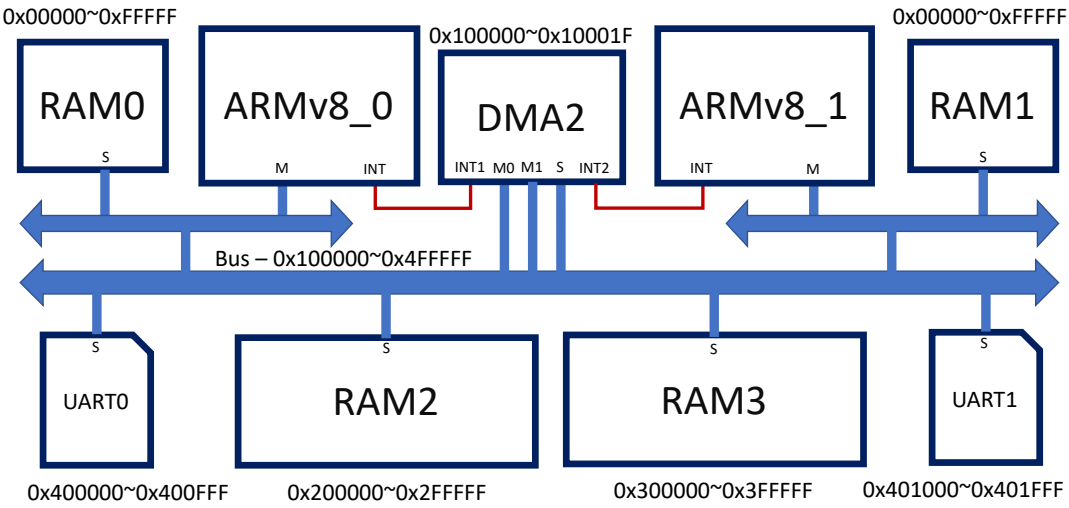

Figure 1 Target Architecture

one (1) `BASE` address. Two (2) master ports are supported, and both ARMv8 cores can use the DMA2 in parallel. It is suggested to have two (2) `SC_CTHREAD`s, each serving an ARMv8. An individual test bench is required to verify this new DMA2.

3. At system initialization, load ARMv8_0 and ARMv8_1 programs onto RAM0 and RAM1, respectively, in a bare-metal fashion. Reset all cells in RAM2 and RAM3 to 0, then use the ARMv8_0 and ARMv8_1 programs to load the designated data into them, as described below.
4. The ARMv8_0 and ARMv8_1 bare-metal programs execute the following data exchange actions:
   A. At initialization, ARMv8_0 writes 1KB data into `0x200400` to `0x2007FF` in a repetition of "`0123456789ABCDEF`" without quotes, i.e. "`0123456789ABCDEF0123456789ABCDEF01`...".
   B. At initialization, ARMv8_1 writes 1KB data into `0x300400` to `0x3007FF` in a repetition of "`FEDCBA9876543210`" without quotes, i.e. "`FEDCBA9876543210FEDCBA9876543210FE`...".
   C. ARMv8_0 monitors the data at address `0x200800`. When the data is `0x0`, ARMv8_1 programs the DMA2's 1st set of control registers to copy 1KB of data from `0x200400` (on RAM3) to `0x300000` (on RAM4).
   D. After the 1KB data is transmitted, ARMv8_0 writes a flag `0x1` to address `0x200800` to indicate the end of data transmission.
   E. ARMv8_1 monitors the data at address `0x200800`. When the data is `0x1`, ARMv8_1 programs the DMA2's 2nd set of control registers to move a 1KB data block starting at `0x300400` (in RAM4) to `0x200000` (in RAM3).
   F. After the 1KB data is transmitted, ARMv8_1 writes a `0x0` flag to address `0x200800` to indicate the end of data transmission.
   G. Repeat the above data exchange/transmission three (3) times and stop.
5. Properly use `printf`, or a low level `write` function, to show the progress of ARMv8_0 and ARMv8_1 programs through UART0 and UART1.

6. Write a less than 5-page report to describe how you implement the system and the interrupt synchronization.
7. Submit your work to Moodle before the project deadline and run a demo in the SoC Lab. In the demo you need to **demonstrate your work using Virtualizer GUI to view ARMv8 code execution and bus transactions**.

Due Date

15:00, May 14th, 2026, at the SoC Lab.

Score weight (towards the final grade) 30%

- 6%: Submit the less than 5-page report
- 12%: Complete the dual-core-DMA2-quad-memory architecture
- 12%
    - 6% to complete both the ISR of ARMv8_1 and ARMv8_2 programs and run
    - 3% to modify DMA and synchronize between two ARMv8 cores
    - 3% to a successful data exchange as expected